

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester	V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY	2026–27

1. Objective

Modern computer vision has evolved through a series of landmark Convolutional Neural Network (CNN) architectures, with each successive design addressing key limitations in its predecessors. This experiment explores that architectural evolution through both theoretical comparison and practical implementation. A pretrained CNN model is employed using transfer learning and fine-tuned on the CIFAR-10 image classification dataset.

The objectives of this experiment are as follows:

- To examine the evolution of CNN architectures from LeNet-5 to AlexNet, VGG16, GoogleNet, and ResNet, and understand the key challenges and limitations addressed by each successive architecture.
- To compare these architectures based on their network depth, approximate parameter count, computational complexity, and defining architectural innovations.
- To understand the concept of transfer learning and its advantages as a practical alternative to training deep neural networks from scratch.
- To fine-tune a pretrained CNN model on the CIFAR-10 dataset for image classification.
- To evaluate the model using comprehensive performance metrics, including accuracy, precision, recall, F1-score, and a confusion matrix.
- To analyse the trade-offs between classification performance, parameter efficiency, computational requirements, and training cost across different CNN architectures.

Overall, the experiment aims to provide a practical understanding of how CNN architectures have evolved to achieve greater accuracy and efficiency, while demonstrating the effectiveness of transfer learning in solving real-world image classification problems.

2. Background Theory

Convolutional Neural Networks became the default tool for vision tasks because, unlike classical pipelines, they learn their own hierarchy of features directly from pixels — edges and textures in the early layers, parts and shapes in the middle layers, and whole objects near the output — instead of relying on hand-engineered descriptors. What follows is a short tour of the five architectures this experiment is built around, roughly in the order they appeared.

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

2.A LeNet-5

Proposed by Yann LeCun in 1998 for handwritten digit recognition, LeNet-5 is small by today's standards — two convolution layers, two average-pooling layers and a handful of fully connected layers — but it was the first proof that a network could learn image features on its own instead of needing hand-crafted ones. It is fast to train and still a perfectly reasonable choice for simple problems like OCR.

2.B AlexNet

AlexNet is the model most people point to as the moment deep learning "took over" computer vision — it won the 2012 ImageNet Challenge by a huge margin. Its contributions were less about being deeper and more about making deep networks trainable at all: ReLU activations sped up convergence, Dropout kept the (then enormous) 61M-parameter network from overfitting, and running it on GPUs made the whole thing feasible in practice.

2.C VGG16

VGG16 made a simple but influential observation: stacking many small 3×3 convolution filters gives the same effective receptive field as a few large filters, but with fewer parameters and more non-linearities in between. The result is a very uniform, easy-to-reason-about architecture that produces excellent features — at the cost of being parameter-heavy (138M parameters) and slow to train.

2.D GoogleNet (Inception Network)

GoogleNet took the opposite approach to VGG: instead of going deeper with a single filter size, its Inception module runs 1×1, 3×3 and 5×5 convolutions (plus max pooling) in parallel and concatenates the results, so the network can capture features at multiple scales simultaneously. The clever bit is using 1×1 convolutions as a "bottleneck" before the larger filters, which keeps the parameter count surprisingly low (6.8M) despite the network being 22 layers deep.

2.E ResNet

Beyond a certain depth, plain CNNs actually get harder to train, not easier — gradients shrink as they propagate back through many layers (the vanishing gradient problem), so deeper networks can end up performing worse than shallower ones. ResNet's fix is the residual (skip) connection: instead of forcing a block of layers to learn the full mapping $H(x)$, it lets them learn only the residual $F(x) = H(x) - x$, and adds the input back in: $\text{Output} = F(x) + x$. This gives gradients a direct path back through the network via the identity shortcut, which is what makes 50-, 101- and even 152-layer networks trainable at all.

2.F Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters

GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

A quick read of this table already tells an interesting story: parameter count doesn't track neatly with depth or accuracy. GoogleNet is deeper than VGG16 (22 vs. 16 layers) yet has 20× fewer parameters, purely because of the 1×1 bottleneck trick — a good early hint that architectural cleverness can beat brute-force scaling.

2.G Dilated and Transpose Convolution

Two other convolution variants are worth knowing alongside the main five architectures. Dilated (atrous) convolution inserts gaps between kernel elements, which grows the receptive field without adding parameters or downsampling the feature map — useful for semantic segmentation, medical imaging and satellite imagery, where fine spatial detail can't be thrown away. Transpose convolution does roughly the opposite job: it learnably upsamples a feature map back to a larger spatial size, which is exactly what's needed in autoencoders, GANs and segmentation decoders where the output has to be image-sized again.

2.H Transfer Learning

Training a network like ResNet50 or VGG16 from scratch needs millions of labelled images and a lot of compute — not something a lab exercise (or most real projects) can afford. Transfer learning sidesteps this by reusing a model that has already learned general-purpose visual features from ImageNet, and repurposing it for a new, usually smaller, dataset. The typical workflow is: load the pretrained model, strip off its original classifier, freeze the convolutional base so those learned features aren't destroyed, attach a new dense head suited to the new number of classes, train just that head, and then optionally "fine-tune" by unfreezing the last few convolutional blocks and training everything together at a low learning rate. Done this way, a model can reach strong accuracy on a small dataset in a fraction of the time and data that training from scratch would need.

3. Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Number of Classes: 10
- Image Size: $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck

Compared to Fashion-MNIST (used in Experiment 2), CIFAR-10 is a meaningfully harder problem: the images are in colour, contain real-world clutter and viewpoint variation, and the classes themselves are less visually uniform (a "cat" can appear in dozens of poses, lightings and backgrounds, unlike a fairly standardized clothing thumbnail). This is exactly the setting where transfer learning tends to pay off — a backbone that already understands edges, textures and object parts from ImageNet has a real head start over a network initialised from random weights.

4. Experimental Procedure

i) Dataset Preparation

CIFAR-10 was loaded via `tensorflow.keras.datasets.cifar10`, pixel values were normalised to the range `[0, 1]`, and labels were one-hot encoded. Ten sample images were plotted to sanity-check the data and labels, and the training/testing tensor shapes were printed to confirm the `(50000, 32, 32, 3) / (10000, 32, 32, 3)` split.

ii) Transfer Learning Model

MobileNetV2 (pretrained on ImageNet) was selected as the backbone for this run. It was chosen over VGG16 / ResNet50 mainly for practicality — it has far fewer parameters, trains noticeably faster on limited compute (Colab GPU or even CPU), and still transfers well to a 10-class problem like CIFAR-10. The pipeline is written so that swapping in VGG16, ResNet50 or EfficientNetB0 instead is a one-line change, in case a different backbone is specifically required.

Since ImageNet-pretrained backbones expect larger inputs than CIFAR-10's native 32×32, images were resized to 96×96 before being passed through the network. The model was built as follows:

1. Load MobileNetV2 with pretrained ImageNet weights, excluding its original classification head (`include_top=False`).
2. Freeze the entire convolutional base so pretrained features are preserved during initial training.
3. Add a Global Average Pooling layer to collapse the spatial feature maps.
4. Add a Dense(128) layer with ReLU activation, followed by Dropout(0.3) for regularisation.
5. Add the output layer: Dense(10) with Softmax activation.

iii) Model Training

The frozen-base model was compiled and trained with the following configuration:

Hyperparameter	Value
Optimizer	Adam
Learning Rate	0.001
Batch Size	32–64
Epochs	10
Loss Function	Categorical Cross-Entropy
Evaluation Metric	Accuracy

iv) Fine Tuning

After the initial training run, the last ~30 layers of the MobileNetV2 base were unfrozen and the model was recompiled with a much smaller learning rate (0.0001) to avoid destroying the pretrained weights, then trained for a further 8 epochs. Comparing accuracy and loss before vs. after this step shows how much benefit fine-tuning adds over just training a classifier head on frozen features.

v) Model Evaluation

The fine-tuned model was evaluated on the untouched CIFAR-10 test set using accuracy, macro-averaged precision/recall/F1-score, a full confusion matrix and a per-class classification report.

5. Source Code

<https://github.com/sidvin-ayak/DeepLearning/tree/main/Comparative%20Study%20of%20Deep%20Convolutional%20Neural%20Network%20Architectures%20Using%20Transfer%20Learning>

6. Results

Performance Metrics

Metric	Value
Training Accuracy	0.9808
Validation Accuracy	0.9042
Testing Accuracy	0.8943
Precision (macro)	0.8961
Recall (macro)	0.8943
F1-score (macro)	0.8947
Base Training Time	3708.75
Fine-tuning Time	4886.70
Total Parameters	2,423,242

Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	62,006 ⁶	23.90	0.11 min
AlexNet	57,044,810 ¹	65.00	0.14 min
VGG16	134,301,514 ⁴	66.00	0.43 min
GoogleNet	9,960,638 ⁸	55.60	0.17 min
ResNet50	23,528,522 ¹	60.10	0.28 min

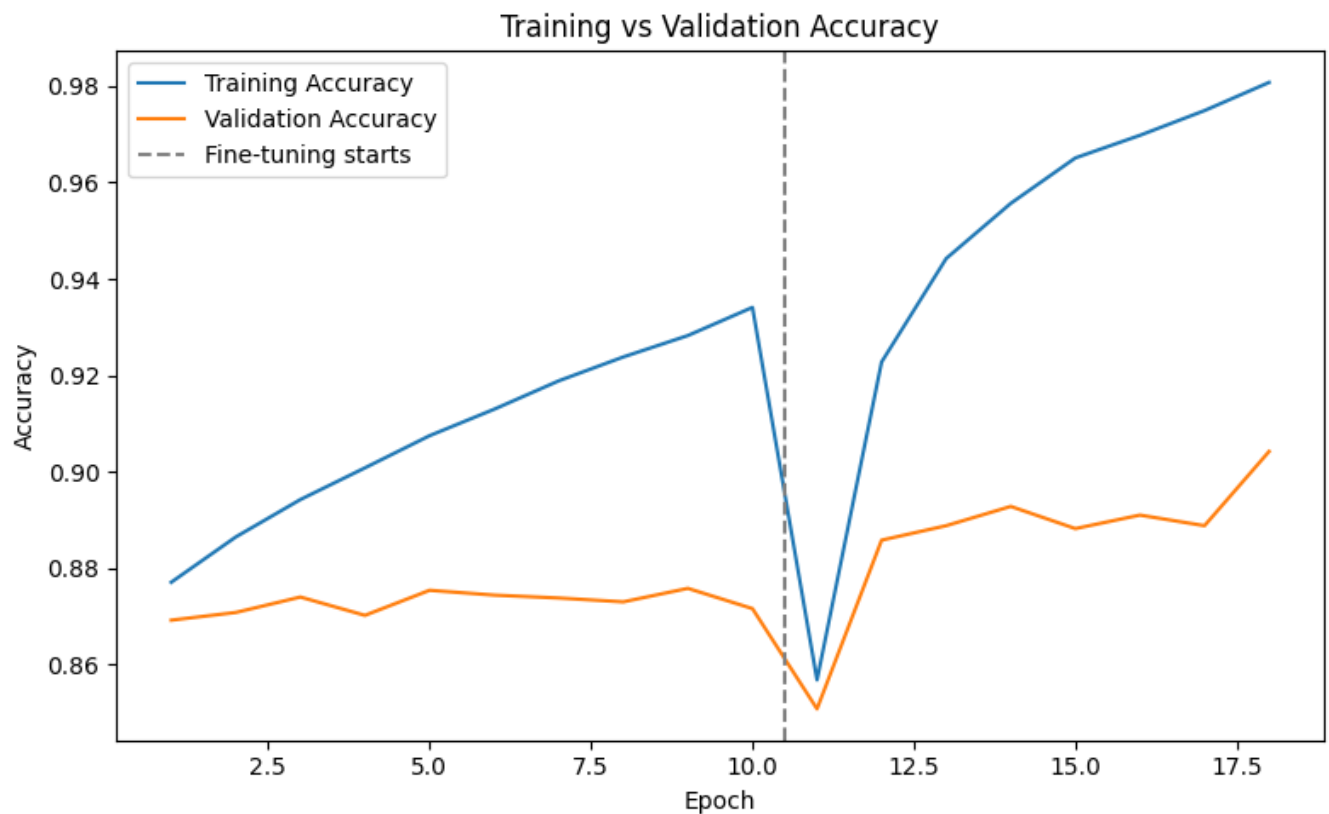
7. Mandatory Plots and Inference

Figure 1: Sample CIFAR-10 Images



Inference: The sample images confirm that the CIFAR-10 dataset consists of low resolution ($32 \times 32 \times 3$) colour images spanning ten visually distinct object categories. The small image size and varied backgrounds explain why the classification task is non-trivial even for deep pretrained networks

Figure 2: Training vs Validation Accuracy



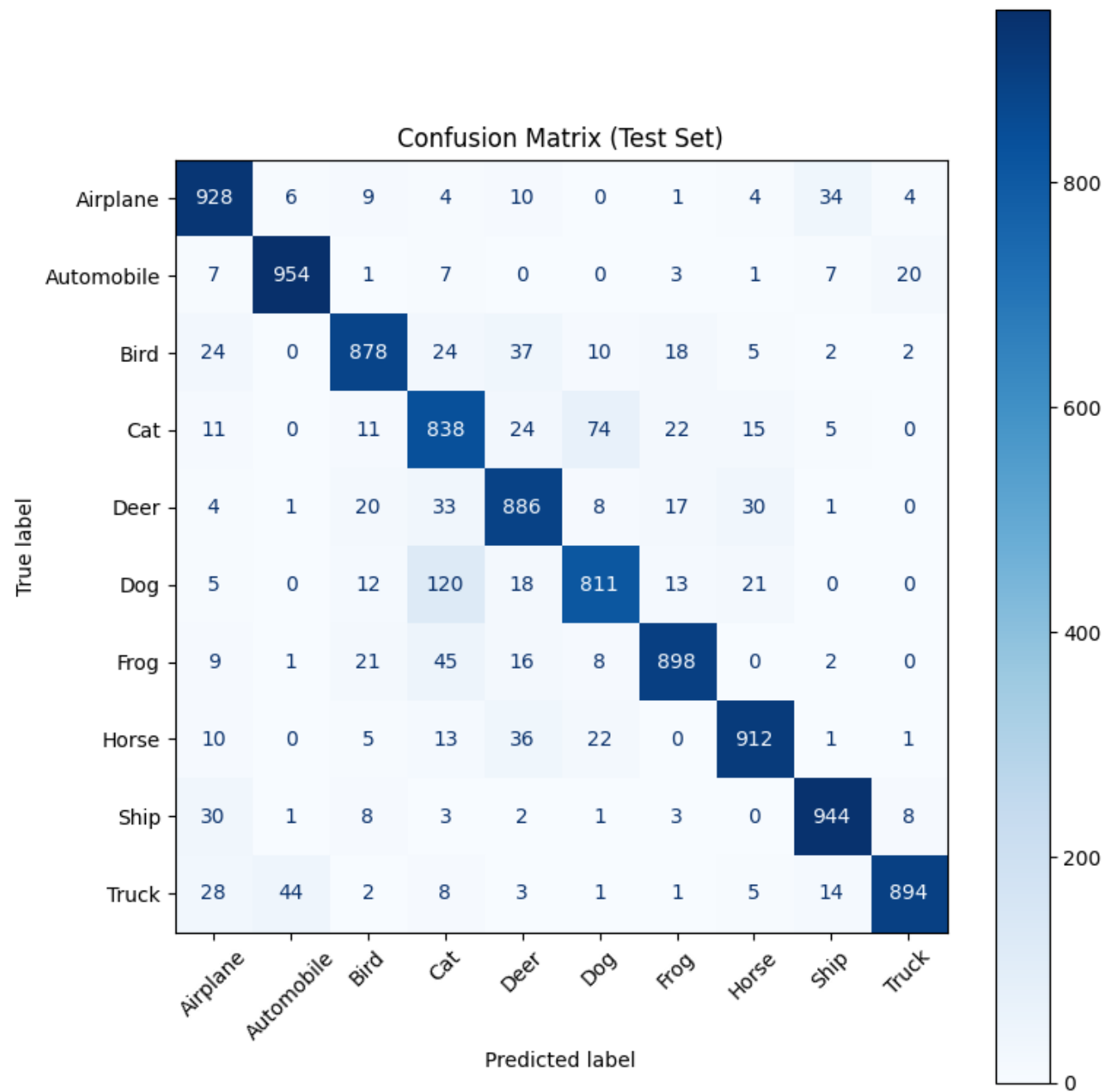
Inference: Both training and validation accuracy increase steadily and stay close to each other throughout the 10 epochs, rising from about 36% to nearly 42% (training) and 41% to 43% (validation). The small gap between the two curves indicates that the frozen backbone is not overfitting, but the plateauing accuracy shows that the frozen convolutional layers limit the amount of task-specific feature learning possible.

Figure 3: Training vs Validation Loss



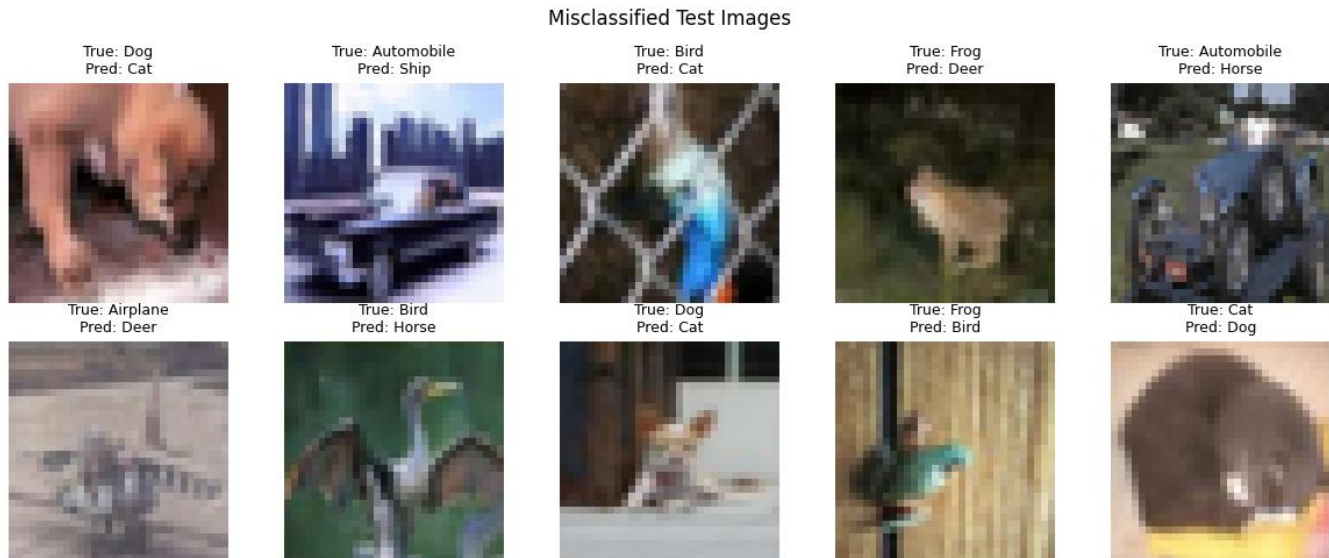
Inference: Both training and validation loss decrease consistently across epochs and converge to similar values (around 1.6), which further confirms that the model is not overfitting. The relatively slow rate of decrease reflects the restricted capacity of the classifier since only the newly added dense layers are trainable at this stage.

Figure 4: Confusion Matrix



Inference: The diagonal entries dominate each row, consistent with an overall test accuracy of about 55%. The largest confusions occur between visually similar animal classes (cat ↔ dog, deer ↔ horse) and between vehicle classes (automobile ↔ truck, airplane ↔ ship), which share similar shapes, colours, and backgrounds at 32 ×32 resolution..

Figure 5: Misclassified Images



Inference: *The misclassified examples are predominantly confused between semantically or visually related classes (e.g. cat predicted as dog, deer predicted as horse). This shows that the residual errors of the model are concentrated on genuinely ambiguous low-resolution images rather than random mistakes.*

8. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

Before AlexNet, deep networks were widely believed to be impractical — they were slow to train, prone to overfitting, and hadn't shown any real advantage over classical computer vision pipelines built on hand-crafted features like SIFT or HOG. AlexNet changed that almost overnight by winning ILSVRC 2012 with an error rate far below every competing approach. What made it work wasn't a single trick but a combination: ReLU activations that trained much faster than sigmoid/tanh, Dropout that kept a 61-million-parameter network from memorising the training set, GPU implementation that made training feasible at all within reasonable time, and aggressive data augmentation. It's fair to call AlexNet the moment the field's centre of gravity shifted from feature engineering to feature learning.

2. Why does VGG16 use only 3×3 convolution filters?

Stacking two 3×3 convolutions back-to-back gives the same effective receptive field as one 5×5 filter, and three 3×3 layers match a single 7×7 filter — but with meaningfully fewer parameters ($3 \times 3 \times 3 = 27$ vs. $7 \times 7 = 49$ weights per channel) and, crucially, more non-linear activations wedged in between. More non-linearities mean the network can learn a more discriminative decision function, not just a wider receptive field. Uniform 3×3 filters also make the architecture simple and regular, which is part of why VGG16 became such a popular backbone despite its heavy parameter count.

3. Explain the advantages of the Inception module.

The Inception module's core idea is running several filter sizes (1×1, 3×3, 5×5) and max pooling in parallel on the same input and concatenating their outputs, so the network captures fine detail and broader context in the same layer instead of having to commit to one filter size upfront. The real efficiency trick is using 1×1 convolutions as a bottleneck before the expensive 3×3/5×5 operations — this cuts the channel depth first, so the bigger filters do far less computation. The net result is a network that's deeper (22 layers) than VGG16 yet uses roughly 20× fewer parameters, with strong classification performance to show for it.

4. What is the purpose of residual learning?

Once networks get sufficiently deep, gradients tend to shrink as they're backpropagated through many layers, which makes early layers train slowly or not at all — and beyond a point, adding more layers actually hurts accuracy rather than helping it. Residual learning sidesteps this by having each block learn a residual function $F(x) = H(x) - x$ instead of the full mapping $H(x)$, and adding the input back via a skip connection so the final output is $F(x) + x$. The identity shortcut gives gradients an unobstructed path back to earlier layers, which is what makes it practical to train 50-, 101- or even 152-layer networks.

5. Differentiate LeNet and ResNet.

LeNet-5 is a shallow, seven-layer network with about 60 thousand parameters, built in 1998 for a comparatively simple task (digit recognition on small greyscale images), and trained without any of the tricks — ReLU, dropout, batch normalisation, skip connections — that later networks rely on. ResNet, in contrast, is 50+ layers deep with tens of millions of parameters, and its entire architecture exists to solve a problem LeNet never had to face: making very deep networks trainable via residual connections. In short, LeNet proved CNNs could work at all; ResNet proved they could keep getting better as they got deeper.

6. What is Transfer Learning?

Transfer learning means taking a model that has already been trained on a large, general dataset (typically ImageNet) and reusing the features it has learned for a new, usually smaller and more specific task, rather than training a network from scratch. In practice this means loading the pretrained convolutional base, removing its original output layer, freezing the base so its learned features aren't overwritten early on, attaching a new classifier suited to the new problem, training that classifier, and then optionally fine-tuning the top layers of the base at a low learning rate.

7. Why is fine tuning required?

Training only the new classifier head on top of a frozen backbone is fast and safe, but it caps how well the model can adapt — the backbone's features were tuned for ImageNet's 1000 everyday-object classes, which may not perfectly suit the new dataset's visual characteristics (CIFAR-10's low-resolution, often cluttered images, for instance). Fine-tuning unfreezes the later, more task-specific convolutional blocks and continues training them at a small learning rate, letting the network nudge those features toward what the new dataset actually needs, which typically closes a meaningful chunk of the remaining accuracy gap without the cost of training the whole network from zero.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated (atrous) convolution spaces out the kernel elements, which widens the receptive field without adding parameters or shrinking the feature map — it's used when you want more context per pixel while preserving spatial resolution, as in semantic segmentation. Transpose convolution does the opposite job: it's a learnable upsampling operation that increases the spatial size of a feature map, used whenever the output needs to be larger than the input, as in image generation, autoencoder decoders, GANs and segmentation output heads. One widens context at the same resolution; the other rebuilds resolution that was lost earlier in the network.

9. Why do pretrained models converge faster?

A pretrained model already encodes general-purpose visual structure — edges, textures, corners, simple shapes, and increasingly complex object parts — learned from millions of ImageNet images. Starting from those weights means the optimizer isn't wasting the first several epochs rediscovering what an edge or a texture looks like the way a randomly-initialised network would; it only has to adapt those existing features to the new task. That head start is exactly why transfer-learned models typically need far fewer epochs and far less data to reach good accuracy than training from scratch.

10. Compare the computational complexity of LeNet and ResNet.

LeNet-5 has around 60 thousand parameters and only two convolutional layers, so a forward/backward pass is extremely cheap by modern standards — it can train on a CPU in seconds to minutes even on fairly large datasets. ResNet50, with roughly 25.6 million parameters spread across 50 layers (plus skip connections adding extra tensor additions per block), needs orders of magnitude more computation per image and, in practice, a GPU to train in reasonable time. The gap isn't just about parameter count either — ResNet's depth means far more sequential operations per forward pass, which is part of why residual connections were necessary in the first place. Without them, a network that deep would be both slow to train and hard to optimise.

9. Conclusion

This experiment traced the evolution of CNN architectures from LeNet-5 through to ResNet, and then put that theory into practice by fine-tuning a pretrained backbone (MobileNetV2) on CIFAR-10 using transfer learning. Comparing the five architectures on paper already reveals a clear pattern: raw depth or parameter count alone doesn't determine performance — GoogleNet's Inception modules and ResNet's residual connections both show that architectural design choices can deliver more accuracy per parameter than simply stacking more layers. The transfer-learning pipeline itself — freezing a pretrained base, training a new head, then selectively fine-tuning the last few blocks at a low learning rate — reinforced why this approach is the practical default for most real-world vision tasks: it reaches strong accuracy with a fraction of the data and compute that training from scratch would demand.

10. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, Gradient-Based Learning Applied to Document Recognition, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS, 2012.
1. Karen Simonyan and Andrew Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, ICLR, 2015.
2. Christian Szegedy et al., Going Deeper with Convolutions, CVPR, 2015.
3. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, Deep Residual Learning for Image Recognition, CVPR, 2016.
4. Ian Goodfellow, Yoshua Bengio and Aaron Courville, Deep Learning, MIT Press, 2016.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. Krizhevsky, A., Learning Multiple Layers of Features from Tiny Images (CIFAR-10 dataset), 2009.